

Secure Programming Application Report

Applying OWASP Secure Coding Practices to a University Registration System

Introduction

This report evaluates a standard software engineering project: a hypothetical *University Course Registration System (UCRS)*. The UCRS is a web application that allows students to register for modules, upload prerequisite documents (such as past transcripts), and interact via course discussion forums. During early development, security was treated as a secondary concern to functional requirements. To rectify this, the project architecture has been audited against the OWASP Secure Coding Practices Quick Reference Guide. Below are five specific recommendations drawn from the OWASP guidelines, detailing the underlying security problems in the system, what needs to change, and how principles from secure software design will be utilized to implement and verify compliance.

1. Input Validation: Validate Data Range (Item 12)

The Problem: In the UCRS, students input the number of credits they wish to register for in a given semester. Currently, the system lacks mathematical bounds validation. A malicious or erroneous user could input a negative number (potentially confusing the billing logic to issue a refund instead of a charge) or an excessively large integer (causing integer overflows or application crashes).

Proposed Change & Verification: The system must enforce strict boundary checks on the server side before any logic is processed. From a secure programming perspective, this is solved using **Design by Contract (DbC)**. The backend controller responsible for processing credits must declare a strict precondition contract (e.g., `0 < credits <= 30`). By enforcing this on the server rather than trusting HTML5 client-side checks, we apply the principle of **Defense in Depth**. To check compliance, developers will employ Test-Driven Development (TDD) by writing explicit unit tests that inject out-of-range boundaries into the application; the pipeline will only pass if the contract actively throws an invalid argument exception.

2. Authentication: Enforce Password Complexity (Item 25)

The Problem: The current registration portal allows users to create student accounts with trivial passwords such as "password" or "123456". This critical flaw leaves student accounts, which contain personally identifiable information (PII) and financial records, highly vulnerable to automated dictionary and brute-force credential stuffing attacks.

Proposed Change & Verification: The application must be redesigned to enforce a strong password complexity policy (minimum 12 characters, requiring uppercase, lowercase, numbers, and symbols). To ensure compliance at a software design level, the system will rely on **Fail-Safe Defaults**. The authentication module's default state must be set to *reject* any account creation attempt unless the password actively matches a predefined strong Regular Expression (Regex) policy. Rather than writing custom security functions, the design will abstract this to a standardized identity library (like Passport.js or Django Auth). We can verify compliance by introducing Static Application Security Testing (SAST) into the repository, configured to flag the absence of strong regex complexity checks in the user creation models.

3. Database Security: Parameterized Queries (Item 129)

The Problem: The UCRS features a search bar that allows students to look up course catalogs. Currently, the backend dynamically constructs SQL queries by directly concatenating the user's search string (e.g., "SELECT * FROM courses WHERE name = '" + userInput + "'"). This creates a direct SQL Injection (SQLi) vulnerability, allowing attackers to terminate the query and append malicious commands to drop tables or exfiltrate private database data.

Proposed Change & Verification: The codebase must replace all dynamic SQL with parameterized queries (prepared statements) or utilize a robust Object-Relational Mapper (ORM). This leverages the secure design concept of **Separation of Domains**, which strictly separates data inputs from executable query logic. To systemically check compliance and enforce this rule, the team will implement **Taint Analysis** techniques within the CI/CD pipeline. The analysis tools will trace all user-supplied data ("tainted" inputs) to ensure it never reaches a database execution sink without first passing through the ORM's sanitization and parameterization boundaries. Any violation will instantly break the deployment build.

4. File Management: Validate Uploaded File Types (Item 138)

The Problem: The prerequisite upload feature allows students to submit documents. Currently, the code attempts to restrict dangerous files by using a "block-list" (e.g., rejecting files ending in .exe or .php). Attackers can easily bypass this by renaming files (e.g., script.php.pdf) or manipulating the HTTP headers, allowing them to upload executable server-side scripts resulting in Remote Code Execution (RCE).

Proposed Change & Verification: The architecture must shift to a **Positive Security Model (Allow-Listing)**. Instead of guessing what to block, the application will solely permit known, safe file formats essential for the business logic (e.g., strictly .pdf or .docx). During the design phase, the validation mechanism will be programmed to inspect the file's "magic bytes" (its internal cryptographic file signature) rather than its superficial extension. Furthermore, to adhere to the **Principle of Least Privilege**, uploaded files will be stripped of execution permissions and stored in an isolated, sandboxed cloud bucket completely outside the web root. Compliance is verified through architectural peer reviews and automated penetration testing targeting the upload endpoint.

5. Output Encoding: Encode User-Supplied Data (Item 168)

The Problem: The system contains a course review forum where students can post feedback. Currently, text entered into the forum is stored in the database and subsequently rendered directly into the HTML of other users' browsers without any sanitization. If an attacker submits a script tag as their review, the browser executes it. This is a classic Stored Cross-Site Scripting (XSS) vulnerability, enabling session cookie theft.

Proposed Change & Verification: The application requires Context-Aware Output Encoding. The core design principle invoked here is **Zero Trust** regarding stored data—data must be neutralized exactly at the point it transitions from the application to the browser interface. The software design will be updated to utilize a modern view-templating engine (such as Jinja2, React, or Angular) that provides **Auto-Escaping** by default. To check compliance, standard coding rules will be enforced via static code linters (like ESLint). These linters will be configured to aggressively flag and block the use of dangerous manual rendering functions like `innerHTML` or `bypassSecurityTrustHtml()`, ensuring developers cannot accidentally circumvent the templating engine's built-in encoding defenses.

Conclusion

Securing the University Course Registration System requires moving beyond reactionary patch management. By directly applying OWASP Secure Coding Practices through the lens of robust software design techniques—such as Design by Contract, Defense in Depth, Fail-Safe Defaults, Allow-Listing, and automated Taint Analysis—the development team can build inherently resilient software. These strategies ensure that security is systematically validated throughout the Software Development Life Cycle (SDLC), mitigating critical flaws before they ever reach a production environment.